

Prefront + Warrant: Feature Specification

2026-09-19 · @Someone

Summary

Prefront is the control plane that makes AI agents safe to run unattended: every tool call an agent makes is authorized before it happens and judged after it happens, from one tamper-evident trace. Enforcement (the Warrant layer) proves each side effect back to a signed human decision and blocks anything outside it. Judgement (the Prefront layer) reads the same trace and finds what authorization alone cannot see: invented parameters, work nobody approved, sessions that drift, agents that are not predictable. Findings flow back into tighter permissions, so the boundary narrows with use instead of widening.

The pitch in one line: **authorize, judge, prove, from one trace**. No other product does all three; identity vendors stop at the token, observability vendors stop at the dashboard, and neither can answer an auditor's question "was this action permitted, was it correct, and show me the evidence."

Policy-document ingestion (turning an enterprise's PDF rulebook into executable rules) is deliberately deferred to a later release; the product below stands without it.

The problem

Enterprises are moving agents from chat assistants to workers that act: they reply to customers, move money, change records and spawn sub-agents, often with no human online. Three failures follow, and today's stack addresses none of them end to end.

Failure	What happens	Why current tools miss it
Unbounded delegation	An agent holds the user's OAuth token and can do anything the user can, for as long as the token lives, with no record of what the user actually asked for	OAuth tokens carry identity and scope, not intent, budget, counterparties or a task boundary

Failure	What happens	Why current tools miss it
Context loss and unpredictability	Across many tool calls the model drops constraints, retries with wider scope, invents parameters, or acts on injected content it read from a web page	Gateways see one call at a time; observability tools log it but do not judge it
No evidence	When something goes wrong nobody can reconstruct who authorized what, what the model was shown, and whether the outcome was inside the rules	Logs are unsigned, scattered across services and keyed on nothing shared

The regulatory clock is also running. The EU AI Act’s human-oversight obligations, SOC 2 access-control criteria and sector rules (HIPAA, banking model-risk guidance) all require an organisation to show that automated actions were permitted and supervised. Today that evidence is assembled by hand, if at all.

OWASP’s 2026 Top 10 for agentic applications names the same gaps directly: excessive agency, insecure tool use, prompt injection through retrieved content, and missing human-in-the-loop controls.

What the product is

Two planes over one trace. The enforcement plane sits on the call path and decides; the judgement plane sits beside it and evaluates. Both key on the same task tree, so one task reads as one story from consent to outcome.

```
flowchart LR
    U[User consent<br/>signed Mission] --> E[Enforcement plane<br/>tokens, PDS, gateway]
    E --> A[Agent runtime<br/>Intent Binder]
    A --> E
    E --> T[Tools / MCP servers]
    E --> TR["Canonical trace<br/>tree_id, node_id, payloads"]
    TR --> J["Judgement plane<br/>integrity, conformance"]
    J --> B["Boundary<br/>narrower Mission + policy"]
    B --> E
```

Reading it: the user's consent becomes a signed Mission; every tool call is checked against it and recorded with its full payload; the judgement plane grades the recorded sessions; its findings become a narrower boundary that the enforcement plane applies next time.

Plane	Question it answers	Timing	Output
Enforcement	Was this action permitted by this user, for this task, within budget?	Per call, on the path, under 3 ms added	allow / deny / step-up, plus a signed record
Judgement	Was the action honest, in envelope, and consistent with what the agent did before?	Per call, per session and per population, off the path	findings, evidence packs, proposed tighter boundary

The design rule that holds it together: **nothing the model outputs can widen its own permissions, and nothing the judgement plane does can delay a call.**

Architecture

Nine components in three trust zones. The control zone holds identity, decisions and hashes only; the data zone holds payloads; the runtime zone is library code inside the customer's agent.

Component	Zone	One job
Consent Component	Runtime (end-user app)	Turn a task into one readable approval: instruction, actions, budget, window, counterparties
Mission Authority	Control	Sign and store Missions; publish keys so any resource server can verify them offline
Token Service	Control	OAuth 2.1 server that stamps <code>mission</code> , <code>tree_id</code> , <code>node_id</code> and actor chain into every token; only path to a downstream token

Component	Zone	One job
Intent Binder	Runtime (agent process)	Sign an Action Attestation per side-effect call: action, argument hash, plan step, evidence hashes; ship the underlying content to the trace
Policy Decision Service	Data path (sidecar)	Deterministic allow / deny / step-up on actor + user + mission + action + resource + arguments
Enforcement Gateway or Tool SDK	Data path	Validate token and attestation, call PDS, capture full request and response, report to the trace; gateway needs no tool changes
Task Control Plane	Control	System of record for the task tree: nodes, decisions, budgets, revocation, denylist, timeline
Evidence Store	Data (tenant side)	Full payloads keyed on <code>node_id</code> and <code>args_hash</code> ; PII stays in the tenant's zone
Evaluation Engine	Data (tenant side)	Judgement plane: provenance graph, intent conformance, population statistics, findings

```

flowchart TD
    subgraph Control zone
        MA[Mission Authority]
        TS[Token Service]
        TCP[Task Control Plane]
    end
    subgraph Runtime zone
        CC[Consent Component]
        IB[Intent Binder]
    end
    subgraph Data zone
        GW[Gateway / Tool SDK]
        PDS[Policy Decision Service]
        ES[(Evidence Store)]
        EE[Evaluation Engine]
    end

```

```
end
CC --> MA --> TS
IB --> TS
IB --> GW --> PDS
GW --> TCP
GW --> ES --> EE
TCP --> EE
```

The split matters for security review: a bank can run the control zone in its own account with hashes only, keep every payload inside its data zone, and still hand an auditor one chain of evidence because the hash in the control zone matches the payload in the data zone.

Feature set: enforcement plane

Every feature here is enforced by a party the model cannot reach. The agent proposes; the PDS and the tool dispose.

Feature	What the customer gets	Buyer question it answers
Signed Missions	One consent screen per task, captured as a signed object: instruction hash, allowed action classes, resources, counterparties, budget, validity window, sub-agent depth	How do I know the agent is doing what the user asked?
Task-tree tokens	Every token in a task carries the same <code>tree_id</code> ; sub-agents get narrower tokens by exchange only; depth is capped; tokens are proof-of-possession bound so a stolen token is useless	Can a sub-agent escalate? Can a leaked token be replayed?
Action Attestations	Each side-effect call carries a signed statement of the action, argument hash, plan step and the evidence the model acted on; tools refuse calls without one	Was this call what the model actually proposed, on what input?

Feature	What the customer gets	Buyer question it answers
Policy Decision Service	Allow / deny / step-up in task vocabulary (<code>email.reply</code> , <code>payment.send</code>), evaluated under 2 ms; built-in checks for attestation match, mission membership, tree liveness and budget	Who says yes or no, and how fast?
Prompt-injection tripwire	Deny high-blast-radius actions whose evidence chain includes untrusted-origin content	What stops a web page from spending my money?
Step-up mid-task	A denied delta (new counterparty, over budget) pauses only that branch, shows the user the delta under the original instruction, resumes on approval; delivered to Slack, Teams, mobile, email	Where do approvals show up?
Headless operation	Standing Missions up to 30 days with workload attestation let scheduled agents run with no user online, inside budget	Can it run at 3 a.m.?
Tree revocation	One call revokes every token in a task; denylist reaches every PDS and gateway within 5 s even when an issuer does not cooperate; anomaly signals (spend velocity, new counterparties, unusual fan-out) can trigger it	Who presses stop?
Gateway mode	Proxy in front of MCP and HTTP tools does token, attestation, PDS and trace capture with zero changes to the tool	Do I have to change my tools?
Tool SDKs	Go, TypeScript, Python, Java; under 50 lines to integrate for teams that want in-process checks	Same, for teams that prefer it in code

Feature	What the customer gets	Buyer question it answers
Runtime adapters	One decorator for Claude Agent SDK, OpenAI Agents SDK, LangGraph, CrewAI, Vercel AI SDK	Does it work with my framework?
Legacy Adapter	Sidecar presenting API-key and cookie services as compliant resource servers; the real credential lives in a per-tenant vault the agent never sees	What about my long tail of internal services?
Trust Broker	Cross-organisation delegation that preserves the task tree and actor chain under a signed bilateral agreement	Can my agent act on a partner's systems?
IdP-first identity	User identity only from Okta, Entra, Google; no second identity store	Is this another directory?
Open verifier	Mission and attestation formats published as an RFC 9396 authorization-details profile with open-source verifiers per language	Why would anyone outside my org trust these tokens?

Verified behaviour: a 51-scenario executable model of the enforcement design passes every case, including replayed tokens, forwarded sub-agent tokens, injected payment instructions, over-budget trees, superseded Missions and cross-org revocation.

Feature set: judgement plane

Authorization says an action was permitted. The judgement plane says whether it was honest, in envelope and consistent, and it does so from day one of trace ingestion with no policy onboarding. Two families of checks, one verdict contract.

Integrity invariants (built in, not configurable away)

Mechanism: a value provenance graph. Every value in every tool call is resolved to its origin (user message, prior tool result, configuration, prompt constant) and classed trusted, semi-trusted or untrusted. Because the runtime library ships the underlying

content, not just hashes, provenance is computed independently of what the agent claimed.

Check	Finding
Fabricated parameter	Argument has no legitimate origin; the agent invented it
Distorted parameter	Origin exists but the value was altered beyond whitelisted transforms
Dropped constraint	An upstream constraint never reached the call: over-broad retrieval
Smuggled value	A value with untrusted origin reached a side-effect call
Stale value	A value was reused after a later tool result superseded it
Evidence mismatch	The runtime's attestation labelled content trusted that provenance shows was retrieved

The last row is the one the enforcement plane cannot do for itself: its injection tripwire depends on the runtime labelling evidence honestly. The judgement plane audits that labelling after the fact.

Intent conformance (against the approved catalogue)

Each approved intent is an envelope: callers, channels, fields, mandatory filter, expected volume, side effect, precondition, ordering, closing obligation, toxic combinations. Actual behaviour is diffed against it at three levels.

Level	Question	Checks
Call	Was this an approved shape?	catalogue membership, field scope, filter scope, volume scope
Session	Was this an approved combination?	toxic combination, goal alignment (task drift vs stated request), workflow integrity (closing obligation abandoned), redundancy (retry storms)

Level	Question	Checks
Population	Is the agent predictable?	outcome consistency (nondeterminism, quantified), invocation drift after each deploy, verdict trend

One verdict contract

Every check speaks the same language: `{ check_id, effect, status: satisfied | violated | indeterminate, evidence }`. Off the path, a verdict is a shadow: divergence from what the agent actually did is the finding, and indeterminate splits into missing precondition versus visibility gap. Per-call integrity checks can later be promoted inline as a PDS plugin; session and population checks stay out of band by nature.

Preflight

Generated test runs against the customer's own tool schemas, so UAT (which has no real traffic) produces findings before production does. Output is capability, not incidence: what the agent could do wrong, ranked.

Feature set: lifecycle loop

Enforcement answers "was this authorized?" Judgement answers "was this right?" The loop turns the second into a better first: the boundary narrows with use.

```
flowchart LR
    D[Derive<br/>capability inventory,<br/>draft Mission + policy] --> T[Test<br/>Preflight + sandbox replay]
    T --> H[Human approves]
    H --> E[Enforce<br/>PDS + gateway]
    E --> P[Prove<br/>evidence chain,<br/>compliance export]
    P --> N[Narrow<br/>effective vs used]
    N --> D
```

Stage	Feature	What it produces
Derive	Boundary Deriver reads tool manifests, system prompt, IdP roles and the action-class registry	Capability inventory (every action the agent could take, tagged by blast radius); draft Mission template; draft PDS bundle with OPA and OpenFGA export
Test	Preflight runs generated probes; sandbox replays a proposed policy against last week's recorded attestations; the judgement plane grades both	Three finding classes: policy would have denied (works), policy would have allowed but Mission did not intend (tighten), no policy covers it (extend the vocabulary). Exit: three consecutive adaptive passes with zero unintended allows
Enforce	PDS in log-only, then advisory burn-in, then enforce intent by intent; break-glass override with recorded justification	Enforced decisions with policy version and test report ID stamped on each record
Prove	Every decision carries the policy bundle version and the test report it was enforced under; export layer maps records to SOC 2 CC6, EU AI Act Article 14, HIPAA 164.312; OCSF events to Splunk, Sentinel, Chronicle; OpenTelemetry traces keyed on <code>tree_id</code>	Evidence packs generated, not assembled
Narrow	Effective-versus-used report compares the capability inventory with what the agent exercised over a window	Proposed narrower Mission template and policy bundle, applied in one step with the sandbox dry-run as safety check

The loop runs before an agent is enabled and again whenever its tools, prompt or configuration change, so the boundary is re-derived and re-tested on every deploy rather than set once and forgotten.

Integration and deployment

Two integrations are mandatory and both are small; everything else is a config choice. There is no log-scraping mode: the product's evidence is only as strong as its trace, so the trace is always captured by our own components.

Integration	Mandatory?	Effort	Why
Runtime library in the agent process (Intent Binder + Consent SDK)	Yes	One decorator in a supported framework	Only place the user instruction, plan and retrieved content exist; root of trust for both planes
Enforcement Gateway in front of tools	Yes, or the Tool SDK	Deploy a proxy; no tool code changes	Where tokens and attestations are checked and payloads captured
Tool SDK in the tool itself	Alternative to gateway	Under 50 lines	For teams that want in-process checks
IdP connection (Okta, Entra, Google)	Yes	Standard OIDC	<code>sub</code> only ever comes from the customer's IdP
Legacy Adapter	Per legacy service	Manifest mapping operations to action classes	Long-tail services without OAuth
Trust Broker	Only for cross-org flows	Bilateral agreement document	Partner delegation

Deployment options

Option	Where	Fit
Hosted	Control zone in our region, data zone (evidence store, evaluation engine, PDS replicas, gateway) in the customer's account	Default for mid-market

Option	Where	Fit
Customer cloud	All zones in the customer's account; Helm chart	Regulated enterprises
Air-gapped	No outbound calls; self-contained key hierarchy	Banks, defence, health

Time to value

Milestone	Target
Quickstart: point at IdP, wrap one agent, first timeline in log-only mode	Under 15 minutes
First judgement-plane findings (integrity invariants need no onboarding)	Same day as first traces
First enforced denial	Same session, by flipping one intent to enforce
Advisory burn-in complete, enforcing by intent	Day 30

Latency added to a tool call by the enforcement plane is budgeted at 3 ms p99 (token validation 0.3 ms, attestation check 0.5 ms, PDS decision 2 ms, trace event asynchronous). The judgement plane adds nothing to the call path by construction; a dead evaluation engine changes no decision.

What it does not do, and what is deferred

Saying this plainly is part of the pitch: the product is narrow where narrowness is a strength.

- It does not shape or filter what the agent sees. No context layer, no semantic layer, no data masking at ingress. The agent's inputs are recorded and judged, not rewritten.
- It does not read policy documents in this release. Rules come from the intent catalogue (derived from tool schemas, prompts and IdP roles) and from built-in invariants. Turning an enterprise's PDF rulebook into executable, human-approved rules is the next major release; the verdict contract already has the slot for it.

- It does not replace the IdP, the gateway vendor or the observability stack. It sits on OAuth 2.1 and the MCP authorization spec, exports OCSF and OpenTelemetry, and imports identity.
- It does not judge model quality (hallucination, tone, helpfulness). It judges whether actions were permitted, honest and in envelope.
- It is not a log-analytics product. There is no OTLP-only mode; evidence comes only from our runtime library and gateway or SDK.

Deferred	Why later	Release
Policy-document ingestion (extract → human review → versioned rule pack)	Needs the trace and catalogue first; adds a second onboarding motion	Next major
Inline promotion of per-call integrity checks into the PDS	Prove them out of band first, then move the deterministic ones onto the path	After policy
Per-user and per-agent aggregate budgets	Needs a second ledger keyed on sub ; schema allows it	Phase 3
Federated broker directory with starter counterparties	Go-to-market problem, not a technical one	Phase 4

Why now, why this

Three things converged in 2026 that did not exist a year earlier: the MCP authorization spec (July 2026) gives agents a standard token flow to extend rather than replace; agent frameworks consolidated to five SDKs, so one decorator each covers most of the market; and enforcement of the EU AI Act's high-risk obligations turned "show me the oversight" from a policy question into an audit question with a date on it.

Competitive framing

Category	What they do	What they cannot answer
Identity and access (Okta, Entra, Auth0, agent-identity startups)	Issue tokens to agents; some add per-agent identity	Whether this call served the user's task, was inside budget, or was driven by injected content; nothing after the token
API and AI gateways (Kong, Apigee, LLM gateways)	Rate limits, key management, per-call content filters	Task-level authorization, sub-agent trees, revocation as a group, anything spanning a session
Authorization engines (OPA, Cedar, OpenFGA)	Evaluate policy fast	Where the policy comes from, whether it was right, and evidence that it was enforced; we export to them
Observability and evals (LangSmith, Arize, Braintrust, Phoenix)	Trace and score model outputs	Enforce anything; verify that traces are complete or untampered; connect a finding to a permission
AI security posture (Zenity-style)	Inventory agents, flag risky configurations	Act at call time; prove a specific action was permitted

The defensible position is the trace itself. Because every record is produced by our runtime library and gateway, signed, and keyed on a task tree, it is the only trace an auditor can trust and the only trace on which judgement can be built without guessing. Competitors would need both the enforcement point and the runtime hook to reproduce it, and each of them owns at most one.

What we ask leadership and investors to believe

1. Enterprises will not deploy acting agents without a party the model cannot reach saying yes or no per call. Gateways and identity vendors agree; none of them own the runtime half.

- 2. Authorization alone is insufficient, because an authorized action can still be dishonest or out of envelope. That gap is where incidents will come from, and it is judged, not enforced.
- 3. Whoever owns the canonical trace owns the compliance export, the policy feedback loop and, later, the policy-document market.

Buyer, motion, pricing

The merged product shifts the first buyer from the Head of AI alone to a Head of AI and CISO pair: enforcement is what the CISO has been asking for, judgement is what the AI platform team needs to ship, and both are satisfied by the same deployment.

Role	Stake	Message	Their objection and our answer
Head of AI / AI Platform (primary)	Owns the agents; blocked at security review	Move agents from demo to production with a control that says yes per call and evidence that it did	"Another layer?" Two small integrations, gateway needs no tool changes, first timeline in 15 minutes
CISO (co-buyer, can block)	Accountable for what agents do to systems	IAM answers who can access what; we answer whether this agent action, for this task, should happen, and stop it if not	"Is this a second identity store?" No: identity comes only from your IdP; we add Missions and task trees on top of OAuth 2.1
CIO (economic sponsor)	Board-level AI programme, too many governance tools	One control plane that enforces and proves, replacing hand-built approval flows and manual evidence gathering	"Too many governance tools." Most inventory or monitor; this one decides and enforces

Role	Stake	Message	Their objection and our answer
Compliance / Internal Audit (urgency driver)	Manual evidence gathering, regulator questions with dates	Every agent action produces a signed chain: consent → boundary → test → decision → outcome, exported to control IDs	"Will my auditor accept it?" Evidence packs mapped to SOC 2 CC6, EU AI Act Article 14, HIPAA 164.312
Platform and security engineers (daily users)	Policy, approvals, incidents	Admin console: agents, standing Missions, live trees, one-click revoke, policy diff and dry-run, findings	"Who writes the policies?" Derived from what you already have, tested before enforced, narrowed by use

Ideal customer

Enterprises moving agents that act (not only answer) into production in regulated or high-consequence settings: financial services, insurance, health, B2B SaaS with payments or customer data. Signal of fit: an agent project stalled at security or compliance review, or a scheduled agent someone wants to run unattended.

Motion

1. Land with one team, one agent, log-only: quickstart under 15 minutes, first integrity findings the same day.
2. Expand by flipping intents to enforce over a 30-day advisory burn-in; step-ups delivered in Slack or Teams.
3. Standardise when the security team owns policy without engineering help and the first evidence pack is accepted by an auditor.
4. Federate when a partner organisation joins through the Trust Broker.

Pricing

Tier	Unit	Includes
Pilot (free)	Sized for one team, one agent	Log-only enforcement, integrity findings, quickstart
Team	Per decision or per task tree, whichever the customer prefers	Full enforcement, judgement plane, admin console, approval delivery
Enterprise	Platform fee plus volume	Self-hosted or air-gapped, SIEM and SOAR export, evidence packs, Trust Broker, Legacy Adapters

Per-tree pricing is recommended as the headline unit: it maps to something the buyer already counts (tasks run), is stable as agents make more calls per task, and rewards us when customers automate more.

Roadmap

Four phases, each with a test a customer can observe. Effort assumes the team that built the existing evaluation engine and the enforcement design; estimates are engineer-weeks and approximate.

Phase	Weeks	Goal	Ships	Exit test
1 Frictionless pilot	1-10	One team enforces on one agent with zero tool changes	Gateway mode (6), tool SDKs (8), runtime adapters (10), IdP-first identity (4), quickstart (3), open verifier draft (8), evidence store, integrity checks over the SDK trace	Quickstart to first enforced denial under 15 minutes; open verifier accepts a token in a server we do not run; first fabricated-parameter finding on real traffic

Phase	Weeks	Goal	Ships	Exit test
2 Team-wide	11-20	Security team owns policy without engineering	Boundary Deriver with OPA and OpenFGA export (8), Preflight and sandbox replay (16), intent conformance at call and session level, admin console (12), approval delivery to Slack, Teams, mobile, email (6), break-glass (3), non-MCP tools and browser-agent bridge (6), action-class registry (4)	A non-engineer writes and dry-runs a policy from a template; step-up approved from Slack; three consecutive Preflight passes with zero unintended allows on a customer agent
3 Enterprise	21-32	Passes a bank's security review	Population-level checks (consistency, drift, trend), evidence chain and compliance mapping (5), SIEM and SOAR export (5), self-hosted and air-gapped (6), anomaly-driven revocation (6), effective-versus-used narrowing (3), per-user budgets	Self-hosted deployment live; SIEM feed live; evidence pack accepted by one auditor; one narrowing applied in production
4 Ecosystem	33+	Tokens mean something between organisations	Trust Broker directory and starter counterparties (8 + partnerships), IETF submission of the Mission profile, inline promotion of integrity checks into the PDS	Two unrelated customers complete a brokered flow; one external resource server verifies Missions natively

Policy-document ingestion (extract, human review, versioned rule pack) is planned as the release after Phase 2, once the intent catalogue and trace are proven with early customers; it reuses the verdict contract and the Boundary Deriver as-is.

The three features that decide adoption are gateway mode, runtime adapters and the open verifier. If only those ship, Phase 1 still works; everything after widens who can buy, not whether it can be used.

Risks and open questions

Risk	Exposure	Mitigation
Mandatory integration slows land	No log-only entry point; a customer whose call path is closed to new components cannot start	Gateway needs no tool changes; runtime adapter is one decorator; quickstart target is 15 minutes. Measure time-to-first-timeline in every pilot and treat over 1 hour as a product bug
Evidence labelling in the runtime	The runtime library could mislabel retrieved content as user-provided, weakening the injection tripwire	Judgement plane computes provenance independently and flags evidence mismatch; library ships signed; PDS can require registered retrievers
Legacy Adapters concentrate credentials	A compromised adapter is a compromised service	One process per tenant, per-tenant vault keys, every read audited; retire adapters as services adopt OAuth
Positioning drift toward observability	Without policy documents, buyers may read the judgement plane as another eval tool	Lead with enforcement; judgement is sold as what makes enforcement tighter, never as a dashboard

Risk	Exposure	Mitigation
Identity vendors move down-stack	Okta or Entra add task-scoped agent tokens	Our value is the pairing of runtime hook, enforcement point and judgement on one trace; publish the Mission profile as a standard so their tokens interoperate rather than compete
Framework churn	A supported agent SDK changes its hook surface	Adapters are thin; conformance suite runs per framework release in CI
Broker adoption is bilateral	Cross-org value needs both sides on a broker	Defer to Phase 4; seed with a payment network or major SaaS as first counterparty

Open questions to settle before Phase 1

- ☐ Who runs the Mission Authority in a hosted deployment: the customer's IdP, us, or both with cross-signing?
- ☐ Do read actions require attestations when the token is user-scoped, or only side effects? (Current design: side effects only; the judgement plane needs read payloads either way.)
- ☐ Default fail-open or fail-closed for reads when the PDS is unreachable? (Current design: closed.)
- ☐ Does a step-up on one branch pause sibling branches that share the same resource?
- ☐ Evidence store retention and PII handling defaults per vertical profile.
- ☐ Headline pricing unit: per tree or per decision, validated with three design partners.